

NLP Assignment 1 Q1 Report

Muhammad Umer Siddiqui
ms08702@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Abdullah Shaikh
as09245@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Arqish
az09714@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Abstract

This report presents implementations of Byte Pair Encoding (BPE) and WordPiece tokenizers developed from scratch for a small multilingual corpus containing English, Urdu, and Chinese. The report describes the preprocessing pipeline, tokenization procedures, pair-selection criteria, and the effect of vocabulary size on the learned subword vocabulary. Experiments were conducted using vocabulary sizes ranging from 250 to 900. The results show that BPE began learning longer subword strings at smaller vocabulary sizes, while WordPiece required a larger vocabulary before producing substantial numbers of longer subword units. Based on the observed results, a vocabulary size of 700 was selected as the final configuration for the dataset.

Keywords

Natural Language Processing, Tokenization, BPE, WordPiece, Multilingual NLP, Subword Tokenization

1 Introduction

The question provided a small multilingual corpus containing English, Urdu, and Chinese text. The task was to implement Byte Pair Encoding (BPE) and WordPiece tokenizers from scratch and experiment with their hyperparameters on the given corpus. The assignment specifically required consideration of the small corpus size, the three different writing systems, code-switching, Unicode representation, and the trade-off between vocabulary size and token fragmentation.

Our implementation used vocabulary size as its only explicit hyperparameter. The algorithms were implemented in `q1.py`, and the resulting vocabularies were stored in `bpe_vocab.txt` and `wordpiece_vocab.txt`. We experimented with vocabulary sizes of 250, 300, 350, 450, 500, 600, 700, 800, and 900 to examine how the learned subword units changed as the vocabulary grew.

2 Preprocessing

The corpus was first normalized using Unicode Normalization Form C (NFC). This provides a canonical representation for characters that can have multiple Unicode representations while preserving their linguistic content. This was particularly important for a multilingual corpus because the tokenizer operates directly on Unicode characters.

The text was then converted to lowercase. This primarily affected the English portion of the corpus, where uppercase and lowercase versions of the same letter would otherwise be treated as different characters. Lowercasing did not produce a comparable change for the Urdu and Chinese portions of the corpus.

Punctuation was separated from surrounding text rather than removed. This allowed punctuation to remain available as tokens while preventing it from being merged directly with adjacent words. Punctuation was identified using Unicode character categories, making this step independent of any particular language or writing system.

Finally, whitespace was normalized to single spaces. This provided a consistent representation of word boundaries and allowed the implementation to use whitespace-based splitting without being affected by repeated spaces, tabs, or line breaks.

It should be noted that whitespace-based splitting behaves differently across the three languages. Unlike English and Urdu, Chinese does not use spaces between words, so the splitting step treats entire Chinese sentences as single units. Since Chinese characters are themselves meaningful units, the subsequent character-level decomposition remains a reasonable approximation for this corpus.

These preprocessing decisions were intended to normalize the corpus without removing linguistic information. In particular, punctuation was retained rather than discarded, and no stemming, lemmatization, stopword removal, transliteration, or language-specific word normalization was performed. Such transformations could alter the frequency distribution of characters and subwords that the two tokenizers use when deciding which pairs to merge.

3 Tokenization Methods

A parent class called `Tokenizer` was implemented to provide functionality shared by both algorithms. It reads the corpus, performs the preprocessing steps, and provides functionality for writing the resulting vocabulary to a file. The BPE and WordPiece implementations inherit from this class and implement their respective tokenization and vocabulary-building procedures.

3.1 WordPiece

The `WordPieceTokenizer` class contains two main methods: `tokenize` and `vocabulary_builder`.

The `tokenize` method initially represents every word as a sequence of individual characters. The first character of each word is stored normally, while every subsequent character is prefixed with `##`. For example, the word `token` is initially represented conceptually as

[t, ##o, ##k, ##e, ##n].

The `vocabulary_builder` method initializes the vocabulary using these character-level tokens. It then repeatedly calculates the frequency of each token and the frequency of every adjacent token pair.

Unlike BPE, WordPiece does not select a pair solely according to its frequency. For two adjacent tokens x and y , the implementation uses the following score:

$$\text{Score}(x, y) = \frac{\text{Freq}(x, y)}{\text{Freq}(x) \text{Freq}(y)}.$$

The pair with the highest score is selected for merging. The two tokens are combined into a new token, with the ## marker removed from the second token. All occurrences of the selected pair are then replaced by the new token, which is added to the vocabulary.

This process continues until the target vocabulary size is reached or no further adjacent pairs are available.

3.2 Byte Pair Encoding

The BPETokenizer class follows a similar iterative procedure but uses a different pair-selection criterion.

The tokenize method splits each word into its individual characters without adding WordPiece-style ## markers. For example, the word token is initially represented as

$$[t, o, k, e, n].$$

The vocabulary is initialized using all unique characters in the corpus. The algorithm then calculates the frequency of every adjacent token pair. The pair with the highest raw frequency is selected:

$$(x^*, y^*) = \arg \max_{(x, y)} \text{Freq}(x, y).$$

The selected pair is concatenated into a new token, and all occurrences of the pair are replaced by the new token. The new token is added to the vocabulary, and the process is repeated until the target vocabulary size is reached.

3.3 Comparison of Pair Selection

The primary algorithmic difference between the two implementations is therefore the criterion used to select pairs for merging. BPE selects the pair with the highest raw frequency, whereas WordPiece normalizes the pair frequency using the frequencies of the individual tokens.

This difference can cause the two algorithms to learn different subword vocabularies even when they are given the same corpus and target vocabulary size. BPE tends to favor pairs that occur frequently in the corpus, while the WordPiece score can favor pairs that occur frequently relative to the frequencies of their individual components.

4 Experiments and Results

The vocabulary size K was varied to examine how increasing the available vocabulary affected the learned subwords. The tested values were 250, 300, 350, 450, 500, 600, 700, 800, and 900.

At $K = 250$, WordPiece did not learn any token longer than one character in the observed vocabulary. In contrast, BPE had already learned longer strings such as word and tokeniz. This indicates that the two pair-selection criteria produced noticeably different vocabularies even at a relatively small vocabulary size.

Increasing K to 300 produced little observable change in the WordPiece vocabulary. BPE, however, learned additional longer

Table 1: Observed effect of vocabulary size on learned sub-words

K	WordPiece	BPE
250	No token longer than one character was observed	Learned longer strings such as word and tokeniz
300	Little observable change	More long strings, including tokenization, process, and ing
350	Similar to $K = 300$	Continued improvement
450	Noticeable improvement, particularly for Urdu	Continued improvement
500	Little observable change	Little observable change
600	Substantial number of English subword strings	Substantial number of complete words
700	Further improvement	Further improvement
800	Continued improvement	Continued improvement
900	Continued improvement	Continued improvement

strings, including tokenization and process, as well as suffixes such as ing. At $K = 350$, the observed results were similar to those at $K = 300$.

At $K = 450$, WordPiece began producing substantially longer tokens, with particularly noticeable changes in the Urdu portion of the vocabulary. BPE also continued to produce longer subword units. The increase to $K = 500$ produced comparatively little additional change in either vocabulary.

At $K = 600$, WordPiece began learning a substantial number of English subword strings. At the same vocabulary size, BPE was already learning a substantial number of complete words. Increasing K to 700 continued to improve the learned vocabularies for both algorithms. Further increases to 800 and 900 continued this trend for WordPiece.

4.1 Multilingual Observations

The multilingual nature of the corpus affected the vocabularies produced by both algorithms. English words are composed from a relatively small alphabet and contain recurring character sequences, allowing frequent sequences to be merged into longer subwords. This is visible in the English examples learned by BPE at relatively small vocabulary sizes.

The Urdu portion of the corpus showed a more noticeable improvement for WordPiece at larger vocabulary sizes. This demonstrates that the choice of vocabulary size can affect the representation of different languages differently within a shared multilingual vocabulary.

Chinese differs from English and Urdu because its writing system does not use alphabetic word formation in the same way. Consequently, the character-level initialization provides a particularly direct representation of Chinese text. The learned Chinese vocabulary should therefore be interpreted in terms of character sequences rather than assuming that every learned token corresponds to an English-style word.

4.2 BPE and WordPiece Comparison

The experiments demonstrate a difference in how the two algorithms use additional vocabulary capacity. BPE began producing longer strings at smaller vocabulary sizes in our experiments, while WordPiece showed a more noticeable increase in longer subword strings at larger values of K . For example, at $K=300$, BPE had already learned complete tokens such as *tokenization* and *language*, while WordPiece had not yet merged any English character pairs at all. This is because BPE's raw frequency criterion merges them early.

The difference follows from their pair-selection criteria. BPE directly maximizes pair frequency, so frequently occurring sequences are merged first. WordPiece instead considers pair frequency relative to the frequencies of the individual tokens. As a result, a pair that is frequent in isolation is not necessarily selected by WordPiece at the same stage at which it would be selected by BPE.

5 Discussion

The experiments show that vocabulary size has a direct effect on the degree of token fragmentation. A small vocabulary limits the number of merges that can be performed and therefore leaves more of the corpus represented by individual characters or short subwords. Increasing the vocabulary size allows additional merges and consequently produces longer learned units.

The results also show that increasing K does not necessarily produce the same degree of improvement at every step. For example, increasing K from 250 to 300 produced a more noticeable change for BPE than for WordPiece, while larger vocabulary sizes produced more substantial changes in WordPiece.

There is therefore a trade-off in selecting K . A very small vocabulary results in greater token fragmentation, whereas a very large vocabulary can allocate vocabulary entries to increasingly specific strings. Since the assignment corpus is small and multilingual, the selected value should provide useful subword units across the different languages rather than simply maximizing the number of learned strings.

Based on the observed results, $K = 700$ was selected as the final configuration. At this value, WordPiece had begun learning a substantial number of longer subword strings, while BPE was producing many complete words and longer subword units. Values above 700 continued the observed trend, but the additional vocabulary capacity was not considered necessary for the purposes of this small corpus.

6 Conclusion

This report implemented BPE and WordPiece from scratch and examined the effect of vocabulary size on their learned subword vocabularies. The two algorithms differed primarily in their pair-selection

criteria: BPE selected the most frequent adjacent pair, while WordPiece used a normalized score based on pair and individual-token frequencies.

Experiments across vocabulary sizes from 250 to 900 showed that BPE began learning longer strings at smaller vocabulary sizes, while WordPiece showed more substantial improvements at larger vocabulary sizes. The experiments also demonstrated that vocabulary size affects different languages differently within a shared multilingual vocabulary.

Based on the observed results, a vocabulary size of 700 was selected for the final configuration because it provided a reasonable balance between vocabulary size and the quality of the learned subword units for the given corpus.